

Which of the following statements is correct?

- ☒ A. A reference is useful when we intend to change the values of actual arguments through the function call.
- ☒ B. A reference is useful when we want to save memory by preventing the creation of large structure variables that are being passed to the function.
- ☐ C. Only A is correct.
- ☐ D. Neither A nor B is correct.

Answer: A

Which of the following statements are incorrect related to malloc() function?

- ☐ A. malloc() calls constructors automatically.
- ☒ B. malloc() returns a void* pointer.
- ☒ C. malloc() allocates raw memory.
- ☐ D. malloc() initializes allocated memory to zero. Garbage

Answer: A or A & D

What will be the output of the following C++ code?

```
#include<iostream>
#include<stdlib.h>
using namespace std;
```

```
class Test
```

```
{ int num1;
public:
```

```
    Test() ✓
```

```
    {
        cout << "Constructor called";
    }
};
```

Class Test // empty class

```
int main()
```

```
{
    Test *t = (Test *) malloc(sizeof(Test));
    return 0;
}
```

3; 1 byte

4 byte
4 byte raw

- ☒ A. Constructor called
- ☒ B. Compilation Error
- ☒ C. Runtime Error
- ☒ D. No Output

Answer: D

A destructor takes _____ arguments.

- ☒ A. One
- ☒ B. Two
- ☒ C. Any number of arguments
- ☒ D. Zero

Answer: D

What is the difference between delete and delete[] in C++?

- ☒ A. delete is used for a single object, while delete[] is used for an array of objects.
- ☒ B. delete[] calls the destructor for each element of the array.
- ☒ C. delete calls the destructor only once.
- ☒ D. All of the above.

Answer: A

Which operator is used to allocate memory dynamically in C++?

- ☒ A. malloc in C
- ☒ B. alloc
- ☒ C. new
- ☒ D. create

Answer:

Which operator is used to deallocate memory allocated using new?

- ☒ A. free() → malloc()
- ☒ B. delete
- ☒ C. release()
- ☒ D. destroy()

Answer: B

What is the main advantage of passing arguments by reference?

- ☒ A. Increases memory usage
- ☒ B. Prevents modification of original data
- ☒ C. Avoids copying and improves performance
- ☒ D. Makes variables global

Answer: C

Which of the following is true about a reference variable?

- ☒ A. It can be NULL. X int *ref = NULL
- ☒ B. It must be initialized when declared.
- ☒ C. It can be reassigned to another variable later. X ref → 3000 4000
- ☒ D. It occupies no memory.

Answer: B int &ref = num

Which memory area stores dynamically allocated objects?

- ☒ A. Stack
- ☒ B. Code Segment
- ☒ C. Heap Test t1: acceptRandom()
dynamic allocated
- ☒ D. Data Segment static int num1

Answer: C

Wrapping of private data in classes in object-oriented programming languages is termed as

- A. Inheritance
- ☒ B. Encapsulation
- C. Polymorphism
- D. Abstraction

Answer

Function Overloading comes under?

- ☒ A. Runtime Polymorphism
- ☒ B. Compile-time Polymorphism
- ☒ C. Inheritance
- ☒ D. Encapsulation

Answer: B

Minor pillars of object-oriented programming language?

- I. Polymorphism ☒
- II. Concurrency ☒
- III. Persistence ☒
- IV. Hierarchy ☒

- A. I and II only
- B. II, III and IV only
- C. I, II, III and IV
- D. III and IV only

Answer:

_____ always describes the outer behavior of an object.

- A. Encapsulation *inner behavior*
- B. Inheritance
- ☒ C. Abstraction
- D. Constructor

Answer: C

Which feature can be implemented using encapsulation?

- A. Data Hiding ☒
- B. Function Overloading ☒
- C. Inheritance ☒
- D. Dynamic Binding ☒

Answer:

Which feature can be implemented using encapsulation?

- A. Data Hiding
- B. Function Overloading
- C. Inheritance
- D. Dynamic Binding

Answer:

Which of the following is known as Data Hiding?

- A. Declaring data members as public ☒
- B. Declaring data members as private and accessing them through member functions ☒
- C. Using global variables ☒
- D. Using friend functions ☒

Answer: B

Which OOP concept allows one class to acquire properties of another class?

- A. Encapsulation
- B. Abstraction
- C. Inheritance ☒
- D. Polymorphism

Answer: C

Which OOP concept allows the same function name to perform different tasks?

- A. Encapsulation ☒
- B. Polymorphism ☒
- C. Abstraction ☒
- D. Aggregation ☒

Answer: B

Which access specifier makes class members accessible only within the class?

- A. public ☒
- B. protected ☒
- C. private ☒
- D. friend ☒

Answer: C

Which of the following best defines Abstraction?

- A. Hiding implementation details and showing only essential features ☒
- B. Acquiring properties from another class ☒
- C. Binding data and methods together ☒
- D. Using multiple constructors ☒

Answer: A

What is the effect of declaring a data member as const in

- ☒ A. It can be modified after initialization
- ☒ B. It cannot be modified after initialization
- ☒ C. It is shared between objects of a class
- ☒ D. It is mutable

Answer: **B**

What is the effect of declaring a member function as const ?

- ☒ A. It can modify the state of an object
- ☒ B. It cannot modify the state of an object
- ☒ C. It is shared between objects of a class
- ☒ D. It is static

Answer:

What is the purpose of the mutable keyword in C++?

- ☒ A. To make data members constant *const*
- ☒ B. To make data members static *static*
- ☒ C. To allow modification of non constant data members in const member function
- ☒ D. To hide data members from outside access $\rightarrow$ *private*

What is the benefit of using a modular approach in programming?

- ☒ A. It makes the code more complex
- ☒ B. It makes the code more difficult to maintain
- ☒ C. It allows for code reuse and easier maintenance
- ☒ D. It reduces the performance of the program

Can a constructor function be constant?

- ☒ A. Yes, a constructor can be declared as const.
- ☒ B. It depends on the programming language being used.
- ☒ C. No, a constructor cannot be declared as const because it needs to initialize the object's members.
- ☒ D. Constructors can be const, but only for static objects.

How are the constants declared?

- ☒ A. const keyword
- ☒ B. #define preprocessor
- ☒ C. both const keyword and #define preprocessor
- ☒ D. \$define

Answer: **A**

function defn $\Rightarrow$ encapsu
function call $\Leftarrow \Rightarrow$ abstraction